

PRISM: FAST ONLINE LLM SERVING VIA SCHEDULING-MEMORY CO-DESIGN

Xingyu Qu

School of Computer Science, Wuhan University
whu-quxy@whu.edu.cn

Tianhao Lin

School of Computer Science, Wuhan University
lintianhao@whu.edu.cn

Yiqi Li

School of Computer Science, Wuhan University
bruceprofession@whu.edu.cn

Zhiyu Chen

Amazon
zhiyuche@amazon.com

Sheng Wang

School of Computer Science, Wuhan University
swangcs@whu.edu.cn

ABSTRACT

Modern online large language model (LLM) services, such as Retrieval-Augmented Generation (RAG) and agent systems, increasingly expose two prominent characteristics: prompt segmentation (e.g., system instructions, retrieved passages, tool outputs) and hotspot skew, where a small set of these segments recurs frequently across user requests. Failing to jointly exploit these patterns could lead to repeated prefill of hot segments and prolonged TTFT, undermining both throughput and user-perceived responsiveness. However, existing work tackles these patterns independently: KV-cache management mainly exploits segment reuse while scheduling reorders requests to improve cache locality, yet neither aligns request admission with KV-cache retention. To address this gap, we first analyze how scheduling and KV-cache management jointly affect TTFT. Guided by this, we present PRISM (Prefix Reuse Optimization Integrated Scheduling and Memory), which co-designs a query-aware scheduler (QAS) with a demand-aware radix tree (DART) to align request admission with exact-prefix KV retention. Our evaluation results show that, versus the strongest baseline, PRISM reduces average per-QPS P99 TTFT by 23.3% and 37.1% while increasing exact-prefix KV-cache hit rate by 5.9 and 12.2 percentage points on 4B and 13B models, respectively.

1 INTRODUCTION

Modern online LLM services increasingly exhibit two prominent characteristics: *prompt segmentation* and *hotspot skew*. First, as chatbots (Touvron et al., 2023; Zhao et al., 2024; Chiang et al., 2024), retrieval-heavy applications (Lewis et al., 2020; Asai et al., 2024; Hsia et al., 2025), and agent systems (Chen et al., 2021; Yang et al., 2024; Xu et al., 2025) are widely deployed online, many prompts are assembled from recurring context pieces, including system instructions, retrieved passages, and tool outputs, followed by request-specific suffixes. We refer to each identifiable reusable context piece as a *segment*. Second, accesses to these segments are often highly non-uniform and temporally local. Recent studies have shown that a small set of hot segments may recur across many requests within short windows (Wang et al., 2025; Li et al., 2025). These properties have motivated two lines of serving optimizations. Systems that optimize KV-cache memory management (Jin et al., 2026; Yao et al., 2025a; Jiang et al., 2026; Kwon et al., 2023; Zheng et al., 2024) exploit prompt structure to improve cache hit rate, while scheduling-oriented systems (Yu et al., 2022; Wu et al., 2023; Agrawal et al., 2024; Dexter et al., 2025) reorder admission and batching to place requests with shared prefix closer in time.

However, optimizing scheduling and KV-cache management independently can leave substantial reuse unrealized under skewed segment arrivals. Scheduling can create prefix locality by reordering

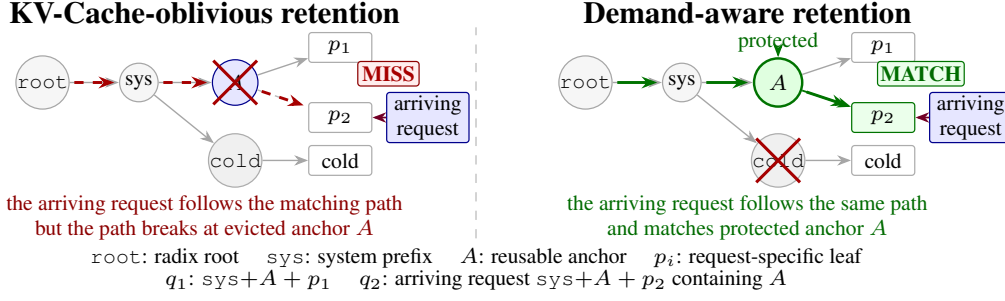


Figure 1: Demand-aware retention in a radix KV-cache tree.

request admission (Dexter et al., 2025; Jiang et al., 2026). Under KV-cache pressure, however, a hot prefix may still be evicted before the next matching request arrives. KV-cache retention should therefore make demand-aware decisions to distinguish hot shared prefixes from cold private suffixes, which come from user input. As Figure 1 illustrates, without demand hints, cold branches may displace anchor A before the next matching request arrives. Instead, demand-aware retention protects A and evicts a cold branch, preserving an exact-prefix hit. This reduces prefill computation and, when coupled with hints from scheduling, lowers TTFT for subsequent hot-segment requests.

This gap leads to three design requirements for online LLM serving. (1) *Segment-level management*: the system should track reusable context pieces rather than only whole requests, because segments are more likely to recur across users or sessions when derived from the same source. (2) *Query-aware scheduling*: the scheduler should use reusable-prefix information to reorder requests, improve KV-cache locality, and pass this information to the KV-cache backend. (3) *Demand-aware KV retention*: under memory pressure, the KV-cache manager should protect prefixes that are active or likely to be reused soon.

To address this gap, we co-design request scheduling and KV-cache management for online LLM serving. We first develop a bottleneck analysis showing how admission control and KV-cache hit rate affect TTFT in different load regimes. Guided by this analysis, we present PRISM (Prefix Reuse Optimization Integrated Scheduling and Memory), a serving architecture with scheduling and memory co-design. PRISM couples a *Query-Aware Scheduler*, which groups arrivals by reusable segments, with *DART (Demand-Aware Radix Tree)*, which uses lightweight scheduler hints to protect high-value shared prefixes and improve cache hit rate.

Overall, our contributions are threefold:

- We provide a bottleneck analysis that links TTFT behavior to admission wait, exact-prefix prefill work, KV pressure, and service-knee behavior on a single GPU, motivating the need to optimize scheduling and KV-cache management jointly.
- We introduce PRISM, which optimizes online LLM serving with request scheduling and KV-cache management co-design.
- The experimental results show strong performance across different datasets and models. PRISM achieves the highest KV-cache hit rate across all methods and reduces average per-QPS P99 TTFT by 23.3% on Qwen3-4B-Instruct-2507 and 37.1% on Llama2-13B.

2 LLM ONLINE SERVING BOTTLENECK ANALYSIS

2.1 LONG-PREFIX REQUESTS AS A COMMON SERVING ABSTRACTION

Many online LLM services share the same structure as Eq. 1 once segments have been serialized into prompt tokens. We index requests by i and denote the i -th request by q_i ¹; its serialized prompt is

$$x_i = [x^{\text{sys}} \parallel x^{r_{i,1}} \parallel x^{r_{i,2}} \parallel \dots \parallel x^{r_{i,m_i}} \parallel x_i^{\text{priv}}], \quad (1)$$

where x^{sys} is a shared system prompt, $r_{i,j}$ is a reusable-segment identity, m_i is the number of reusable segments in q_i , and x_i^{priv} is the request-specific suffix, which depends on user input.

¹Appendix A consolidates the principal theory and algorithm notation used throughout the paper.

2.2 TTFT DECOMPOSITION AND THE VALUE OF PREFIX REUSE

For request q_i , we decompose TTFT as

$$\text{TTFT}_i = W_i^{\text{admit}} + T_i^{\text{prefill}} + T_i^{\text{1tok}}, \quad (2)$$

where W_i^{admit} is the admission wait from the arrival of q_i to the backend prefill launch, T_i^{prefill} is the prefill interval that builds missing KV states, and T_i^{1tok} is the time from prefill completion to the first returned token. In our traces, W_i^{admit} subsumes scheduler-side waiting, backend enqueue, and waiting for GPU service, so the optimization target is jointly minimizing admission delay and effective prefill work.

Let z_i denote the fully serialized token path of request q_i , and let $\mathcal{C}_t$ denote the KV-cached prefix paths resident when q_i is admitted at time t . Because causal transformers can safely reuse KV states only along exact token-prefix matches, the reusable hit length is

$$L_i^{\text{hit}}(t) = \max_{c \in \mathcal{C}_t} \text{LCP}(z_i, c), \quad (3)$$

where LCP counts the longest common prefix tokens. Writing $L_i^{\text{eff}}(t) = \ell(z_i) - L_i^{\text{hit}}(t)$ for the effective prefill length, with $\ell(\cdot)$ denoting token length, we approximate prefill time as

$$T_i^{\text{prefill}} \approx \frac{\ell(z_i) - L_i^{\text{hit}}(t)}{R_{\text{pf}}(B_t)}, \quad (4)$$

where $R_{\text{pf}}(B_t)$ is the realized prefill throughput under batch state B_t . Segment identities are only control-plane hints; KV hits still require exact serialized-prefix equality. This makes the low-load regime immediate: when admission wait is small, improving exact prefix-path hits lowers the dominant prefill term. It also exposes the systems coupling: scheduler decisions help only if overlapping requests are admitted close enough in time for the KV-cache to preserve the relevant prefix paths.

2.3 COMPUTATION AND KV-MEMORY PRESSURE ON A SINGLE GPU

The data plane is constrained by both prefill compute and KV-cache capacity. Eq. (4) captures the compute side: when admission wait is small, reducing the missing prefix length directly reduces prefill work. The memory side creates a separate bottleneck because KV-cache usage grows linearly with live prefix length. Under the normalized RAG budget used throughout the paper ($k=5$ with nominal 128-token chunks), the reusable evidence payload is about 640 tokens before prompt wrappers and private text; under Llama2-13B geometry with FP16 KV states, that payload alone corresponds to roughly 0.5 GiB of KV memory before accounting for the system prompt, user suffix, or concurrent decode state. Thus an online engine cannot simply keep every repeated prefix resident once model weights and runtime buffers are included. It must spend limited KV-cache capacity on prefixes likely to be reused soon; Appendix D reports the realized prompt-length and backend KV-accounting details for the executable traces.

2.4 QUEUEING IMPLICATION AND OPERATING POINTS

We summarize the queueing consequence needed by the design. A *wave* is the set of requests launched together for prefill. Let S_{wave} denote the corresponding wave-duration service time, $\bar{M}_\pi$ the mean wave size, L_π the mean prompt length, L_{reuse} the mean reusable-token payload, h_π the stationary full-prompt token-level exact-prefix hit rate, and $R_{\text{pf},\pi}^{\text{eff}}$ the measured effective prefill throughput under policy π . Since a wave releases $\bar{M}_\pi$ request completions after one wave-duration prefill,

$$\mathbb{E}_\pi[S_{\text{wave}}] = \frac{L_\pi(1 - h_\pi)}{R_{\text{pf},\pi}^{\text{eff}}}, \quad \mu_\pi := \frac{\bar{M}_\pi}{\mathbb{E}_\pi[S_{\text{wave}}]} = \frac{\bar{M}_\pi R_{\text{pf},\pi}^{\text{eff}}}{L_\pi(1 - h_\pi)}. \quad (5)$$

Here $\mathbb{E}_\pi[\cdot]$ denotes expectation under policy π , μ_π is a request-level rate in requests/s, and the $\bar{M}_\pi$ factor is the wave-to-request conversion. For two policies π_1, π_0 ,

$$\frac{\mu_{\pi_1}}{\mu_{\pi_0}} = \frac{\bar{M}_{\pi_1}}{\bar{M}_{\pi_0}} \cdot \frac{R_{\text{pf},\pi_1}^{\text{eff}}}{R_{\text{pf},\pi_0}^{\text{eff}}} \cdot \frac{L_{\pi_0}}{L_{\pi_1}} \cdot \frac{1 - h_{\pi_0}}{1 - h_{\pi_1}}. \quad (6)$$

Thus the general comparison separates batching, realized hardware throughput, prompt length, and exact-prefix reuse. For an offered request arrival rate λ , write $\rho_\pi = \lambda/\mu_\pi$ for utilization. The bulk-service queue is only approximated by an $M/G/1$ model in Appendix E, where we also state the decode-contention and crossover assumptions. The fixed-parameter corollary below isolates the hit-rate term:

Theorem 1 (Fixed-parameter service-rate gap and stability expansion). *Under Eq. (5), consider a controlled comparison where $L_\pi = L$, $\bar{M}_\pi = \bar{M}$, and $R_{\text{pf},\pi}^{\text{eff}} = R_{\text{pf}}$ for PRISM and the LRU baseline. If the realized full-prompt token-level exact-prefix hit-rate gap $\Delta h := h_{\text{PRISM}} - h_{\text{LRU}}$ is nonnegative, then*

$$\mu_{\text{PRISM}} - \mu_{\text{LRU}} = \mu_{\text{LRU}} \cdot \frac{\Delta h}{1 - h_{\text{PRISM}}} \geq 0, \quad (7)$$

with equality iff $\Delta h = 0$. Since the approximate $M/G/1$ stability condition requires $\rho_\pi < 1$, the request-level sustainable arrival rate equals μ_π requests/s and the policy with the larger realized hit rate has the larger stability region, strictly so when $\Delta h > 0$, with expansion

$$\Delta \lambda^* := \lambda_{\text{PRISM}}^* - \lambda_{\text{LRU}}^* = \frac{\bar{M} R_{\text{pf}}}{L} \cdot \frac{\Delta h}{(1 - h_{\text{LRU}} - \Delta h)(1 - h_{\text{LRU}})}. \quad (8)$$

If results report a reusable-only hit-rate gap Δh_{reuse} , the full-prompt gap used above is $\Delta h = \Delta h_{\text{reuse}} L_{\text{reuse}}/L$.

Theorem 1 isolates the algebraic contribution of exact-prefix reuse: once scheduling and retention realize a larger exact-prefix hit rate, the hit-rate term in Eq. (6) raises the service rate and expands the stability ceiling. The queueing approximation in Appendix E then gives the design picture. At low load, admission wait is small and TTFT is dominated by the prefill term $L_\pi(1 - h_\pi)/R_{\text{pf},\pi}^{\text{eff}}$; near the service knee, the queueing factor amplifies each point of Δh . The single-A800 calibration in Appendix D places the Qwen3-4B-Instruct-2507 $k=5$ transition between 50 and 60 offered QPS: PRISM sustains 48.3 req/s at 50 QPS, then plateaus near 50 req/s while P99 TTFT grows from 2.00 s to 6.50 s. The Llama2-13B validation remains a lower-throughput regime, with the previously instrumented crossover around 12–15 QPS.

3 METHODOLOGY: PRISM

3.1 OVERVIEW

We present PRISM, an online LLM serving architecture that reduces TTFT by co-optimizing scheduling and KV-cache management through a Query-Aware Scheduler (QAS) and a Demand-Aware Radix Tree (DART). To optimize cache efficiency, QAS tracks reusable-segment counters across the request lifecycle, covering queuing, candidate batching, and serving. It uses the resulting segment priorities to generate order-sensitive bucket signatures, grouping requests with the same reusable segments into hot lanes. To avoid starvation, QAS also reserves cold-lane slots in each batch. This demand telemetry is shared with DART, which integrates reusable-segment anchors into the radix KV-cache. DART then uses dispatch-batch priorities as a retention and eviction heuristic under memory contention. Appendix B gives the full pseudocode of QAS and DART. Figure 2 shows the system overview.

3.2 QAS: QUERY-AWARE SCHEDULING

State update and segment priority. QAS operates on short pending windows of duration Δt_{win} . After applying completion events, snapshotting the waiting set $\mathcal{Q}^{\text{queue}}$, and observing the active backend set $\mathcal{Q}_t^{\text{active}}$, it maintains three segment counters. For reusable-segment set $\mathcal{R}(q)$,

$$g_t(r) = |\{q \in \mathcal{Q}^{\text{queue}} : r \in \mathcal{R}(q)\}|, \quad a_t(r) = |\{q \in \mathcal{Q}_t^{\text{active}} : r \in \mathcal{R}(q)\}|.$$

For any reference batch in serving $\mathcal{B}$, it also defines

$$n_t(r; \mathcal{B}) = |\{q \in \mathcal{B} : r \in \mathcal{R}(q)\}|.$$

We refer to these quantities as the global, active, and next counters, respectively. The next counter captures segment reuse within a candidate next batch, providing a lightweight near-future demand

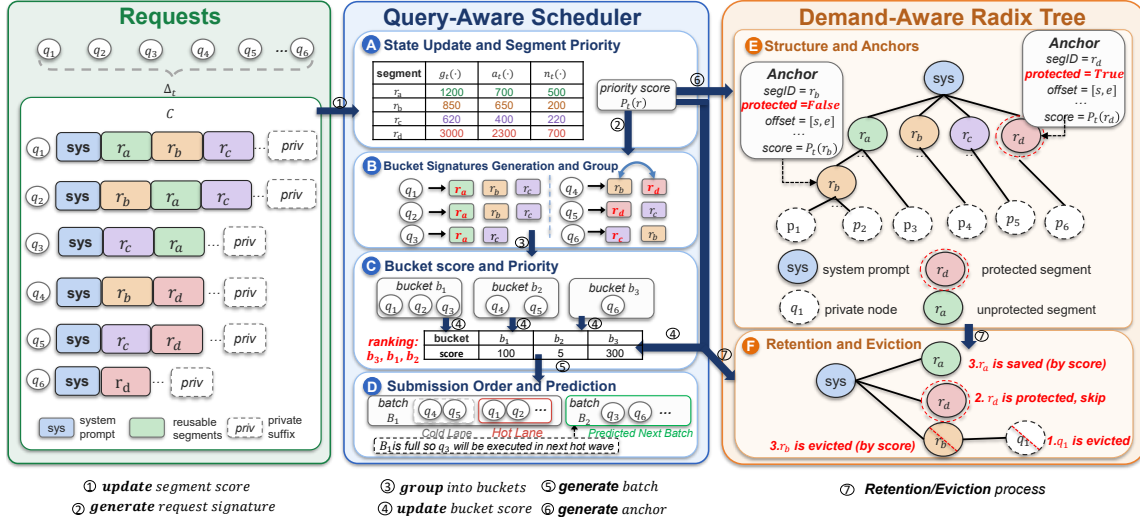


Figure 2: Overview of PRISM.

signal. These raw counts are used for within-round ranking, not for comparing absolute scores across windows. The shared priority score is

$$P_t(r; \mathcal{B}) = w_g g_t(r) + w_a a_t(r) + w_n n_t(r; \mathcal{B}). \quad (9)$$

We use $w_a = 10^6, w_n = 10^5, w_g = 1$ in all experiments. The weights enforce a strict ordinal hierarchy: active demand > future-batch reuse > queued prevalence, so that retention prioritizes active and imminent reuse.

At each QAS round, the three counters are updated and segment priorities are recomputed, providing the scores used by both scheduling and DART retention. Front alignment and signature construction use $P_t(r; \emptyset)$; candidate bucket scoring later instantiates $\mathcal{B}$ as the provisional hot-lane batch $\mathcal{B}_b^{\text{hot}}$, and DART retention instantiates it as the actual dispatch batch $\mathcal{B}_t^{\text{dispatch}}$.

Bucket-signature generation and grouping. Each request arrives with an ordered reusable-segment skeleton $\Pi_i = (r_{i,1}, \dots, r_{i,m_i})$, where m_i is the number of reusable segments. In this stage, QAS first moves the top- f_{front} segments by priority to the front of the reusable-segment skeleton. The reordered skeleton Π_i^* is then used to construct a signature for bucket assignment. For signature size κ , QAS forms an order-sensitive bucket signature

$$\sigma_i = \text{Top}_{\kappa}^{\rightarrow}(\Pi_i^*; P_t(\cdot; \emptyset)), \quad b_i = \text{hash}(\sigma_i). \quad (10)$$

Here, $\text{Top}_{\kappa}^{\rightarrow}$ selects high-priority segments and writes them back in service-time order. Thus two requests share a bucket only when their dominant reusable segments appear in the same serialized order. We use $f_{\text{front}} = 3$ and $\kappa = 1$ in all evaluations. This setting balances prefix construction and grouping granularity: larger f_{front} reorders lower-ranked, less stable documents, adding overhead and risking KV-cache hits, while smaller values form shorter shared prefixes; larger κ fragments buckets and weakens locality.

Bucket score and priority. QAS assigns a priority score to each bucket. Writing $\mathcal{R}_b$ for the reusable segments in bucket b , the bucket utility and score are

$$U(b) = \frac{1}{\max(1, |\mathcal{R}_b|)} \sum_{r \in \mathcal{R}_b} P_t(r; \mathcal{B}_b^{\text{hot}}). \quad (11)$$

This mean utility measures the average reusable-segment priority represented by the bucket, while the raw bucket-size term below captures how many queued requests share the signature.

$$\text{Score}(b) = \alpha_{\text{size}} |\mathcal{T}[b]| + \beta_{\text{util}} U(b), \quad (12)$$

where $|\mathcal{T}[b]|$ is the number of queued requests in bucket b . We set $\alpha_{\text{size}} = 1, \beta_{\text{util}} = 0.5$ in all evaluations.

Submission order and prediction. QAS determines the bucket submission order while leaving the backend batch-size policy unchanged. QAS splits every batch into a hot lane and a cold lane. The hot lane is filled by scanning buckets in descending $\text{Score}(b)$ order to improve KV-cache locality. The cold lane admits q_{cold} cold requests in FIFO order to avoid starvation. As shown in Section 4.5, $q_{\text{cold}} = 0$ maximizes KV-cache reuse but inflates P99 TTFT due to rare-prefix starvation, and larger quotas preserve SLO fairness at the cost of hot-lane capacity. Meanwhile, QAS estimates which pending requests are likely to enter the next batch according to Eq. (12), and uses them to update the next counter.

3.3 DART: DEMAND-AWARE RADIX TREE

DART keeps the standard radix tree over raw token prefixes, annotates selected boundary nodes with QAS metadata, and uses the dispatch-batch priority to decide which resident KV-cache anchors should survive memory pressure. These annotations affect only retention and eviction order.

Tree structure and anchors. DART places anchors at token boundaries after the system prompt, each reusable segment, and the private suffix. These anchors are DART’s added metadata layer over the native radix tree. Each records the segment type, optional reusable-segment identifier, serialized-prompt offset, latest QAS counter snapshot, ownership $\text{seg}(u)$ for evictable reusable nodes, and last-access time $\text{last}(u)$. If an exported offset falls inside a radix edge, DART splits the radix node so that the anchor coincides with an exact token boundary. After QAS fixes $\mathcal{B}_t^{\text{dispatch}}$, DART instantiates the dispatch-batch priority from Eq. (9):

$$\hat{P}_t(r) := P_t(r; \mathcal{B}_t^{\text{dispatch}})$$

Each evictable node is assigned an ownership and protection label: private suffix, unprotected reusable node with score $\hat{P}_t(\text{seg}(u))$, or protected reusable anchor. These labels determine the first coordinate of the eviction key in Eq. (13), while the dispatch-batch score orders unprotected reusable nodes.

Score-guided retention and eviction. DART forms a protection set $\mathcal{F}_t$ from the top- f resident reusable anchors under $\hat{P}_t$, where f is the protection budget. This protects resident anchors with the strongest demand in the current dispatch batch. When memory must be reclaimed, DART orders evictable nodes by

$$\text{EvictKey}(u) = \begin{cases} (0, \text{last}(u)), & u \text{ is a private-suffix node,} \\ (1, \hat{P}_t(\text{seg}(u)), \text{last}(u)), & u \text{ is a reusable node with } u \notin \mathcal{F}_t, \\ (2, \text{last}(u)), & u \in \mathcal{F}_t \text{ is a protected reusable anchor,} \end{cases} \quad (13)$$

where smaller keys are evicted first and $\text{last}(u)$ is the node’s most recent access time. Thus Eq. (13) first reclaims cold private suffixes, then unprotected reusable nodes in ascending dispatch-batch priority, and only then protected reusable anchors. Section 4.5 evaluates this KV-cache-retention dimension directly.

3.4 COMPLEXITY AND OVERHEAD

PRISM’s control path operates on reusable-segment IDs and radix-anchor metadata rather than tokens. Let $S = \sum_{q_i \in Q_{\text{queue}}} m_i$ be the queued reusable-segment occurrences in one scheduling round, and let V be the number of evictable radix nodes. QAS spends expected $O(S)$ time for counters, bucket insertion, and candidate scoring, plus $O(S \log f_{\text{front}})$ for front alignment and $O(S \log \kappa)$ for ordered signatures. DART inherits the backend radix lookup and split costs; its added work is anchor-metadata maintenance and $O(\log V)$ per candidate eviction. PRISM therefore avoids all-pairs semantic search, offline KV precomputation, and per-token control logic, keeping scheduler–KV-cache coordination much cheaper than model prefill and decode execution.

4 EVALUATION

In this section, we first describe how the serving traces are constructed and how their hotspot structure relates to real RAG workloads. We then evaluate whether PRISM converts reusable prompt structure into lower TTFT on online RAG and agent-style workloads, calibrate the service knee predicted by

the bottleneck analysis, and isolate the contribution of DART’s scheduler-informed retention policy. Additional hyperparameter sensitivity and prompt-length variants are reported in Appendices C and F.

4.1 EXPERIMENTAL SETUP

Hardware. All experiments run on a single NVIDIA A800 80GB PCIe GPU attached to a 14 vCPU Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz and use CUDA 13.0. With limited compute resources, we focus on the single-GPU setting.

System Implementation. PRISM is implemented on SGLang (Zheng et al., 2024) with the original continuous batching and chunked-prefill mechanisms retained. QAS refines the scheduling policy on top of SGLang, and DART extends the backend radix tree.

Baselines. We compare against three competitive baselines: **SGLang**, **k-LPM** (Dexter et al., 2025), and **ContextPilot** (Jiang et al., 2026). All methods share the same engine binary, model, arrival process, concurrency limit, and memory settings.

- **SGLang:** native SGLang is the serving substrate of PRISM, so it provides the engine-level baseline with continuous batching, chunked prefill, and radix-prefix KV-cache reuse.
- **k-LPM:** k-LPM is a prefix-reuse scheduling baseline that groups requests by shared prefix structure, making it the closest scheduler-side comparison to QAS. We set $k = 2$ (with the best performance) across all evaluations.
- **ContextPilot:** ContextPilot is a context-reuse serving baseline. To the best of our knowledge, ContextPilot is the state-of-the-art method for prefix reuse scheduling.

Models. We evaluate **Qwen3-4B-Instruct-2507** (Yang et al., 2025a) and **Llama2-13B** (Touvron et al., 2023), representing compact and memory-intensive single-GPU serving regimes.

Metrics. We evaluate the performance and accuracy by:

- **Latency.** We report TTFT percentiles at P50/90/95/99, emphasizing P99 TTFT because online serving systems are often judged by tail responsiveness and SLO violations rather than only average delay.
- **KV-Cache reuse.** We report backend exact-prefix KV-cache hit rate to measure whether scheduling decisions translate into actual KV reuse.
- **Answer quality.** Since the main workload is RAG, we report answer F1 as a correctness guardrail.

4.2 WORKLOAD CONSTRUCTION AND TRACE REALISM

Online serving trace. The main trace is set in RAG scenario because RAG is widely deployed in online LLM serving with patterns of segmentation and hotspot skew. The RAG trace is derived from **MultiHopRAG** (Tang & Yang, 2024). Each request is materialized with a fixed prompt template, a system prefix, $k=5$ retrieved 128-token chunks treated as reusable segments, and a request-specific suffix.

Hotspot and arrival process. We construct the main `zipf_hotspot` trace over the 981 unique evidence passages. The hotspot set $\mathcal{R}_{\text{hot}}$ contains $|\mathcal{R}_{\text{hot}}|=20$ reusable passages, the hot-request rate is $r_{\text{hot}}=0.7$, the Zipf exponent is $\alpha=1.2$, and the replay length is $N=2,048$. A request is hot if at least one retrieved segment belongs to $\mathcal{R}_{\text{hot}}$. To separate offered-load variation from reusable-segment popularity, request timestamps are sampled from a Poisson process at the target QPS, while hotspot documents are sampled according to the Zipfian distribution. Thus the Poisson process models online request arrivals, and the Zipfian component models the skewed recurrence of reusable context segments. Every run starts from an empty KV-cache. All methods receive the same materialized prompts, sampled arrival timestamps, engine binary, model, memory configuration, and empty-cache initialization.

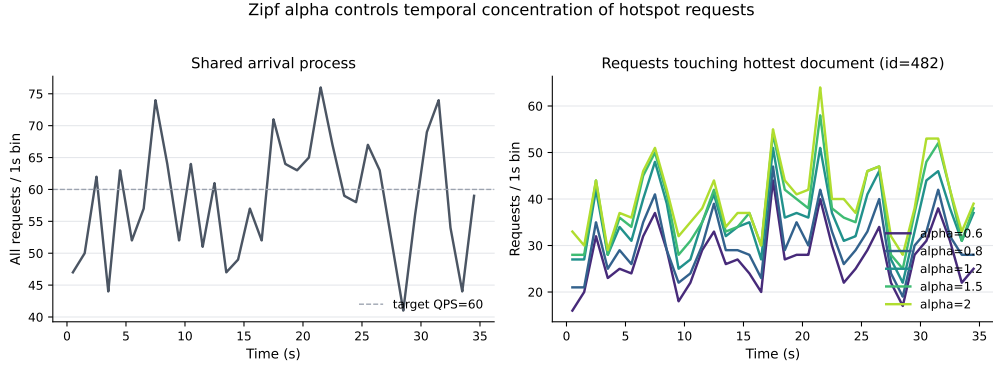


Figure 3: Zipfian hotspot workload structure at 60 QPS. The left panel shows the shared arrival process over 1-second bins, while the right panel shows requests touching the hottest reusable document under different Zipf exponents.

Relation to real traces. Figure 3 visualizes the trace construction used for the Zipf-exponent sensitivity study. The overall arrival stream remains fixed around the target load, while larger α concentrates more requests on the same reusable document. This mirrors the hotspot skew and short-window recurrence observed in real-world RAG serving traces (Wang et al., 2025; Li et al., 2025): a small subset of evidence or context segments can receive disproportionate demand over short windows. The Zipfian model therefore serves as a controlled stress test for the operating condition PRISM targets, rather than as a claim that production traces follow one exact parametric law. Appendix C varies τ_{hot} and α , while Appendix F varies retrieval top- k to change prompt length. Section 4.6 further evaluates agent-style workloads using τ -bench (Yao et al., 2025b).

4.3 MAIN RESULTS

We sweep Qwen3-4B-Instruct-2507 over $\lambda \in \{40, 50, 60, 70, 80\}$ requests/s and Llama2-13B over $\lambda \in \{10, 15, 20, 25, 30\}$ requests/s. Based on Section 2, the first point for each model is pre-knee, while the remaining points cover the transition and finite-replay stress regimes. Figure 4 reports TTFT trends, and Table 1 reports KV-cache reuse and answer quality over the same sweeps.

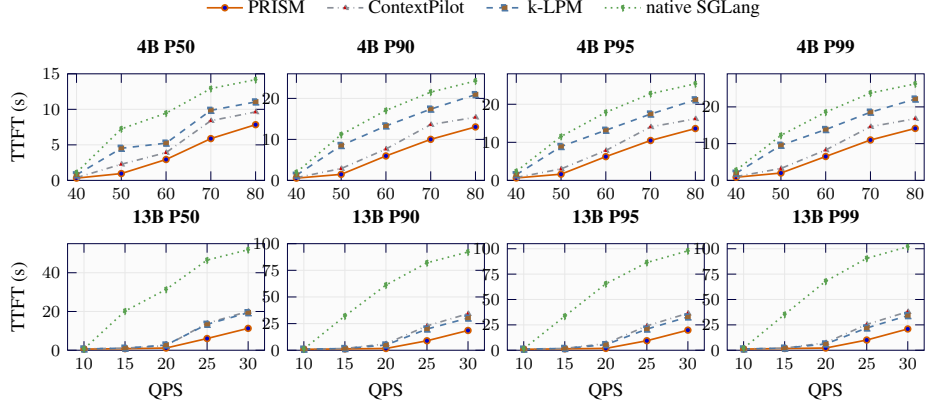


Figure 4: TTFT percentiles versus offered load. Panels report P50/P90/P95/P99 in seconds for 4B and 13B.

Table 1: Per-QPS exact-prefix KV-cache hit rate and answer F1 score.

Model	QPS	PRISM		ContextPilot		k-LPM		native SGLang	
		KV-Cache Hit Rate (%)	F1 Score (%)	KV-Cache Hit Rate (%)	F1 Score (%)	KV-Cache Hit Rate (%)	F1 Score (%)	KV-Cache Hit Rate (%)	F1 Score (%)
Qwen3-4B-Instruct-2507	40	48.98	40.61	43.64	40.33	19.94	39.81	15.17	39.76
	50	48.90	40.33	43.51	39.85	16.93	39.81	15.07	39.81
	60	49.25	40.29	43.76	39.36	20.26	39.76	15.14	39.81
	70	49.10	40.44	42.04	39.48	20.39	39.76	14.98	39.53
	80	49.27	40.28	43.24	40.56	19.86	39.81	16.06	39.48
Llama2-13B	10	39.79	21.69	26.51	20.97	20.10	21.01	10.17	20.78
	15	39.02	21.58	27.01	20.98	20.14	20.45	10.07	20.88
	20	39.15	21.36	27.06	20.48	20.34	20.56	10.14	20.74
	25	39.22	21.79	28.14	20.67	19.89	20.68	9.98	20.96
	30	40.15	21.88	27.50	20.58	20.86	20.66	11.06	20.98

Latency and KV-cache reuse. PRISM gives the lowest TTFT at all reported percentiles. Averaged over the QPS sweep, it reduces P99 TTFT by 23.3% on Qwen3-4B-Instruct-2507 and 37.1% on Llama2-13B relative to the strongest baseline; relative to native SGLang, the reductions are 63.5% and 80.8%. The same runs show consistently higher exact-prefix KV-cache hit rate: about 49.1% versus 43.2% on Qwen3-4B-Instruct-2507, and about 39.5% versus 27.2% on Llama2-13B against the strongest baseline. These results support the bottleneck analysis: near the service knee, small differences in realized prefix reuse are amplified by queueing.

Answer quality. Table 1 shows that F1 remains stable over the sweeps, with PRISM showing a slight improvement. Thus, PRISM’s TTFT gains come from serving-side reuse rather than degraded generation quality.

Robustness. In Appendix C, we evaluate PRISM when r_{hot} and α vary. The results show that PRISM can improve KV-cache hit rate and lower tail TTFT across various workloads, rather than depending on a specific hotspot distribution.

4.4 SERVICE-KNEE CALIBRATION

Figure 5 instantiates the service-demand terms in Eq. (5) on the Qwen3-4B-Instruct-2507 main configuration. For the $k=5$ trace, PRISM follows the offered load through 50 QPS, then saturates near 50 req/s; beyond that point, P99 TTFT and the admission-to-first-token envelope grow rapidly. The fixed-load top- k panel provides the complementary prompt-length stress check: increasing the materialized prompt from 768 to 1029 and 1430 tokens lowers throughput and raises P99 TTFT, as expected from the $L(1 - h_\pi)$ term in the bottleneck model. Thus the observed 50–60 QPS transition is a hardware- and workload-specific operating point rather than a universal constant, and it explains why small hit-rate differences in Table 1 become large tail-latency differences near saturation. Appendix D gives the corresponding trace semantics and backend accounting details.

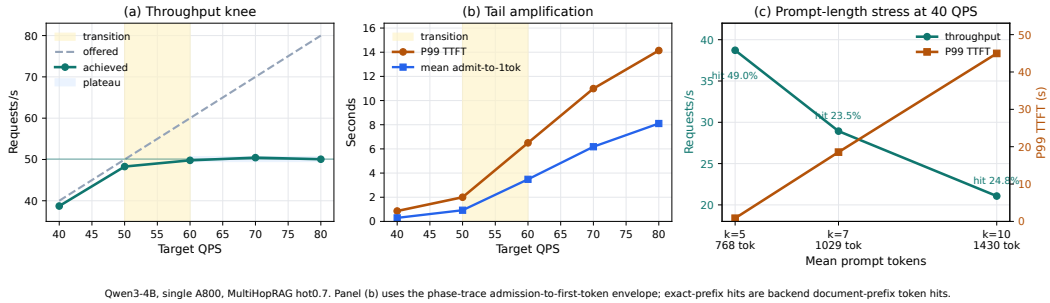


Figure 5: Qwen3-4B-Instruct-2507 service-knee calibration on a single A800 ($k=5$, $r_{\text{hot}}=0.7$), with a fixed-load prompt-length stress check. Panel (a) shows achieved throughput flattening near 50 req/s; panel (b) shows the corresponding tail-latency amplification; panel (c) shows that increasing the materialized prompt length lowers sustained throughput and raises P99 TTFT even at fixed offered load.

4.5 ABLATION STUDY

Since DART presents a new policy of KV-cache retention and eviction, it is natural to ask how other strategies perform with the QAS scheduler. We do not isolate QAS because without QAS hints, DART reduces to LRU. To isolate DART, we keep QAS, the workload, engine binary, and memory configuration fixed, and replace only the backend retention rule with LRU, LRU plus the active-demand counter used by QAS (Algorithm 1), or LFU. The ablation uses Qwen3-4B-Instruct-2507 with the same $k=5$ and 128-token RAG budget, sweeping $\lambda \in \{60, 70, 80\}$ requests/s where KV-cache pressure makes retention choices affect tail latency.

Figure 6 shows that scheduler-informed retention protects more reusable prefix mass than generic policies: PRISM reaches 49.10–49.27% KV-cache hit rate, while the strongest non-PRISM variant, LRU+Active Counter, remains at 30.41–30.56%. Because generic recency, frequency, and active-demand counters cannot see which prefixes QAS will admit next, the KV-cache-side gap reduces P99 TTFT by 37.6%, 15.9%, and 25.4% at 60, 70, and 80 QPS relative to the best non-PRISM baseline.

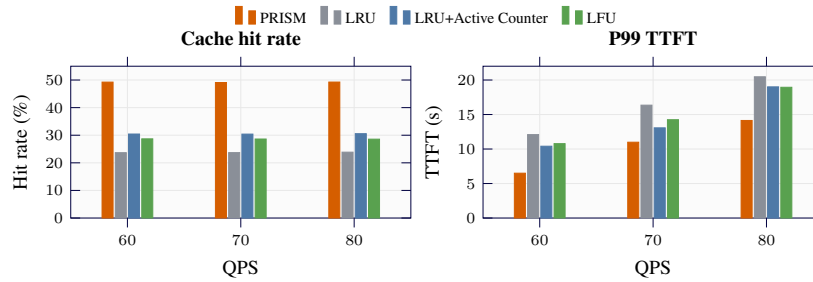


Figure 6: Backend KV-cache-policy ablation: exact-prefix hit rate (left) and P99 TTFT (right).

Cold-lane quota. The FIFO cold lane in QAS exposes a separate latency–reuse tradeoff. We ablate the quota over $q_{\text{cold}} \in \{0, 2, 4, 8\}$ on Qwen3-4B-Instruct-2507, using the same $k=5$, $r_{\text{hot}}=0.7$, Zipf exponent, Poisson arrival process, replay length, and QPS sweep as the main benchmark. The setting $q_{\text{cold}}=0$ disables the FIFO starvation guard, while larger values reserve more dispatch slots for the oldest unselected requests and reduce hot-lane capacity.

Table 2: Cold-lane quota ablation on Qwen3-4B-Instruct-2507. Each entry reports P50/P90/P95/P99 TTFT in seconds; bold marks the best P99 at each QPS.

QPS	$q_{\text{cold}}=0$	$q_{\text{cold}}=2$	$q_{\text{cold}}=4$	$q_{\text{cold}}=8$
<i>Entry format: P50/P90/P95/P99.</i>				
40	0.37/0.79/0.94/1.47	0.32/0.55/0.64/0.86	0.35/0.59/0.67/ 0.85	0.34/0.56/0.66/0.89
50	1.01/1.57/1.73/2.68	0.96/1.47/1.63/2.00	0.93/1.46/1.63/ 1.97	1.03/1.63/1.78/2.03
60	3.09/6.16/6.43/7.60	2.94/5.96/6.24/ 6.50	2.94/6.12/6.46/6.85	3.11/6.34/6.62/7.07
70	5.82/10.24/10.70/12.23	5.88/10.01/10.48/ 10.99	6.12/10.39/10.78/11.29	5.95/10.29/10.72/11.26
80	7.57/12.86/14.53/16.21	7.84/13.04/13.62/ 14.14	7.77/13.20/13.79/14.35	7.39/13.08/13.59/14.81

Table 3: Cold-lane quota ablation on Qwen3-4B-Instruct-2507. Entries report KV-cache hit rate in percent; bold marks the best hit rate at each QPS.

QPS	$q_{\text{cold}}=0$	$q_{\text{cold}}=2$	$q_{\text{cold}}=4$	$q_{\text{cold}}=8$
40	49.87	48.98	49.01	48.89
50	49.79	48.90	48.85	48.82
60	50.07	49.25	49.12	49.05
70	50.28	49.10	49.06	49.18
80	50.30	49.27	49.04	49.30

Tables 2 and 3 show that maximizing reuse alone is not sufficient. Removing the cold lane gives the highest KV-cache hit rate, averaging 50.06% versus 49.10% for the default $q_{\text{cold}}=2$, but it also raises average P99 TTFT from 6.90 s to 8.04 s over the sweep because rare-prefix requests can wait behind hot-prefix buckets. Larger cold quotas preserve the starvation guard but spend more dispatch budget on FIFO traffic: compared with $q_{\text{cold}}=2$, $q_{\text{cold}}=4$ and $q_{\text{cold}}=8$ increase average P99 TTFT by 2.4%

and 4.6%, respectively. We therefore use $q_{\text{cold}}=2$ as a small fairness reserve while leaving most slots available for prefix-local hot-lane batching.

4.6 GENERALIZATION TO AGENT WORKLOADS

We next test whether PRISM’s scheduler-memory co-design extends beyond retrieval-augmented prompts to agent-style prompts. Agent workloads expose reuse not only through retrieved evidence, but also through domain policies, tool schemas, database records, deterministic tool observations, and partially shared dialogue context. These regions are naturally serialized into the prompt of a tool-using agent and therefore expose exact-token prefix structure that a serving system can exploit without changing the model input.

Workload and methods. We construct a serving-side AGENTPREFIX trace from τ -bench (Yao et al., 2025b). Each request preserves the serialized agent prompt, including system and policy text, tool definitions, task-specific records, prior turns, and deterministic tool outputs. The workload should be read as a serving benchmark for agent prompts rather than as an end-to-end measurement of task success. We use the same single-A800 SGLang setup and Qwen3-4B-Instruct-2507 model as the main 4B evaluation. The trace contains 2,048 requests with Poisson arrivals at 30, 40, and 50 QPS, a 32-token decode budget, and a maximum prefill length of 40,960 tokens. PRISM is instantiated in an agent-mode profile that uses the same exact-prefix KV-cache accounting and demand-aware retention mechanism, but chooses locality signatures from hot reusable agent segments rather than from RAG documents. The baseline set matches the main evaluation.

Table 4: AGENTPREFIX serving results on Qwen3-4B-Instruct-2507. TTFT columns report seconds. Hit and NG (Not Global) hit denote total and non-global exact token-prefix hit rates, respectively.

QPS	Method	P50	P90	P95	P99	Hit	NG hit
30	PRISM	21.41	36.41	38.95	41.27	0.764	0.753
30	native SGLang	57.53	105.06	111.37	131.66	0.730	0.642
30	k-LPM	21.60	41.79	43.90	54.96	0.740	0.677
30	ContextPilot	21.39	39.76	42.50	58.88	0.742	0.679
40	PRISM	30.40	52.40	55.10	57.86	0.764	0.751
40	native SGLang	65.53	120.19	127.47	140.71	0.731	0.645
40	k-LPM	30.27	55.76	59.73	94.09	0.741	0.678
40	ContextPilot	30.28	55.42	59.56	65.70	0.741	0.678
50	PRISM	35.42	62.07	65.36	68.68	0.764	0.751
50	native SGLang	72.05	130.46	138.21	150.86	0.731	0.646
50	k-LPM	36.36	64.64	69.12	91.01	0.742	0.678
50	ContextPilot	36.98	65.19	69.68	75.07	0.741	0.676

Table 4 shows that PRISM’s advantage is not specific to segment-based RAG prompts. PRISM has the lowest P90–P99 TTFT at every offered load. Relative to the strongest baseline at each QPS, it reduces P99 TTFT by 24.9%, 11.9%, and 8.5% at 30, 40, and 50 QPS, respectively; relative to native SGLang, the average P99 reduction is 60.7%. The KV-cache metrics show the same mechanism as in the main benchmark: PRISM consistently realizes higher exact-prefix reuse, especially on non-global agent-specific regions, where its hit rate remains around 0.75 compared with roughly 0.68 for the strongest scheduling baselines. These results support the intended scope of PRISM: the system does not assume that reusable prefixes correspond to retrieved passages, but only that near-future demand over repeated, token-identical prompt regions can be estimated at admission time.

5 RELATED WORK

Segmented LLM workloads. Modern LLM services increasingly serve prompts assembled from reusable context pieces rather than from a single monolithic user query. Public chatbot traces and evaluation platforms expose repeated system instructions and interaction templates (Zhao et al., 2024; Chiang et al., 2024); RAG systems attach retrieved evidence to each query (Lewis et al., 2020; Asai et al., 2024; Hsia et al., 2025); and tool-using agents serialize policies, tool schemas, environment observations, and dialogue history into long prompts (Chen et al., 2021; Yao et al., 2023; Schick et al., 2023; Yang et al., 2024; Xu et al., 2025; Yao et al., 2025b). These workloads motivate serving systems that reason about reusable prompt regions. PRISM focuses on the online serving consequence of this

structure: repeated segments are useful only when they are admitted close enough in time and remain resident as exact-prefix KV states.

Online LLM serving engines. General-purpose LLM serving systems optimize batching, memory layout, and execution scheduling. Orca (Yu et al., 2022) introduced iteration-level scheduling for generative inference; FastServe (Wu et al., 2023) studies distributed inference scheduling; FlexGen (Sheng et al., 2023) targets high-throughput offloading on limited GPUs; Sarathi-Serve (Agrawal et al., 2024) uses chunked prefill to manage the throughput–latency tradeoff; vLLM (Kwon et al., 2023) introduces PagedAttention for KV-memory management; and SGLang (Zheng et al., 2024) provides structured-program execution with radix-prefix caching. These systems form the execution substrate for efficient serving.

Serving systems for RAG and agents. RAG serving introduces additional system pressure because retrieved context can be long, repeated, and workload-dependent. PipeRAG (Jiang et al., 2024), RAGO (Jiang et al., 2025), RAGCache (Jin et al., 2026), CacheBlend (Yao et al., 2025a), TurboRAG (Lu et al., 2025), UBIS (Lai et al., 2025) and RAGGED (Hsia et al., 2025) study RAG-specific execution, caching, cached knowledge fusion, vector search, or performance characterization. Agent workloads raise a related but broader reuse problem: repeated tool schemas, policies, database records, and deterministic observations can create exact-prefix opportunities across requests and workflow steps (Yang et al., 2024; Xu et al., 2025; Yao et al., 2025b; Pan et al., 2025). PRISM is designed for both settings by treating retrieved passages and agent components uniformly as reusable segments whose near-future demand can be estimated at admission time.

Reuse-aware scheduling. Scheduling can improve prefix locality by grouping related requests before they reach the backend. k-LPM (Dexter et al., 2025) groups requests by shared prefix structure under latency constraints, while ContextPilot (Jiang et al., 2026) improves exact-prefix reuse through context ordering and scheduling. These methods primarily act on admission order: they can increase the chance that nearby requests share a prefix, but they do not explicitly tell the backend which radix-tree anchors should survive under KV-cache pressure. PRISM differs by making scheduler-side segment counters part of the cache-control plane.

KV-cache reuse, compression, and retention. KV-cache techniques reduce repeated prefill work by retaining, reusing, compressing, or sharing cached states. vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) provide system-level KV memory and prefix-cache mechanisms; CacheGen (Liu et al., 2024) compresses and streams KV states; RAGCache (Jin et al., 2026) caches reusable knowledge for RAG; CacheBlend (Yao et al., 2025a) fuses cached context with new context; KVShare (Yang et al., 2025b) explores semantic-aware sharing; HotPrefix (Li et al., 2025) studies hotness-aware prefix sharing; TurboRAG (Lu et al., 2025) precomputes hot/cold KV chunks offline; and KVFlow (Pan et al., 2025) exploits workflow-level prefix reuse in multi-agent systems. These systems broaden the design space for cached inference, but many rely on offline materialization, semantic approximation, or cache-side policies that are not directly tied to the scheduler’s imminent dispatch. PRISM intentionally stays within exact-prefix reuse and protects radix-tree anchors using online dispatch-batch demand, avoiding semantic false positives while aligning cache retention with near-future admission.

6 CONCLUSION

In this paper, we presented PRISM, an online LLM serving architecture that jointly optimizes request scheduling and KV-cache management for segmented and skewed workloads. Guided by a bottleneck analysis of TTFT, PRISM couples a reusable-segment scheduler QAS with a demand-aware KV-cache manager DART. Across RAG and agent workloads, this scheduling-memory co-design improves KV-cache hit rate and reduces tail TTFT. This work provides a framework for analyzing serving bottlenecks under a specific workload, system, and hardware setting, and points to future opportunities for reducing latency through scheduling-memory co-design.

Limitations and future work. PRISM currently targets fixed hardware and workload settings, so extending the online LLM serving analysis into a more general theory remains future work. Moreover, all our experiments run on a single A800 GPU, leaving multi-GPU KV placement, interconnect effects, and heterogeneous GPU pools to future study.

REFERENCES

- A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134. USENIX Association, 2024.
- A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- W. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. Jordan, J. E. Gonzalez, and I. Stoica. Chatbot Arena: An open platform for evaluating LLMs by human preference. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 8359–8388. PMLR, 2024.
- G. Dexter, S. Tang, A. Fatahi Baarzi, Q. Song, T. Dharamsi, and A. Gupta. LLM query scheduling with prefix reuse and latency constraints. *arXiv preprint arXiv:2502.04677*, 2025.
- J. Hsia, A. Shaikh, Z. Z. Wang, and G. Neubig. RAGGED: Towards informed design of scalable and stable RAG systems. In *Proceedings of the 42nd International Conference on Machine Learning*, pp. 24139–24155. PMLR, 2025.
- W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska. PipeRAG: Fast retrieval-augmented generation via algorithm-system co-design. In *Proceedings of the 15th ACM Symposium on Cloud Computing (SoCC 2024)*. Association for Computing Machinery, 2024.
- W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu. RAGO: Systematic performance optimization for retrieval-augmented generation serving. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA 2025)*. Association for Computing Machinery, 2025.
- Y. Jiang, Y. Huang, L. Cheng, C. Deng, X. Sun, and L. Mai. ContextPilot: Fast long-context inference via context reuse. In *Proceedings of the 9th Conference on Machine Learning and Systems*, 2026.
- C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *ACM Trans. Comput. Syst.*, 44(1):2:1–2:27, 2026.
- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626. ACM, 2023.
- Yuhui Lai, Shixun Huang, and Sheng Wang. Updatable balanced index for stable streaming similarity search over large-scale fresh vectors. In *2025 IEEE International Conference on Big Data (BigData)*, pp. 540–549. IEEE, 2025.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, 2020.
- Y. Li, R. Gu, C. Huan, Z. Wang, R. Yao, C. Tian, and G. Chen. HotPrefix: Hotness-aware KV cache scheduling for efficient prefix sharing in LLM inference systems. *Proc. ACM Manag. Data*, 3(4):250:1–250:27, 2025.

- Y. Liu, H. Li, Y. Cheng, S. Ray, Y. Huang, Q. Zhang, K. Du, J. Yao, S. Lu, G. Ananthanarayanan, M. Maire, H. Hoffmann, A. Holtzman, and J. Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pp. 38–56. ACM, 2024.
- S. Lu, H. Wang, Y. Rong, Z. Chen, and Y. Tang. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 6588–6601. Association for Computational Linguistics, 2025.
- Z. Pan, A. Patel, Z. Hu, Y. Shen, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding. KVFlow: Efficient prefix KV-Caching for accelerating LLM-based multi-agent workflows. *arXiv preprint arXiv:2507.07400*, 2025.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, pp. 31094–31116. PMLR, 2023.
- Y. Tang and Y. Yang. MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. *arXiv preprint arXiv:2401.15391*, 2024.
- H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Z. Wang, Y. Hu, J. Ye, Z. Chang, J. Yu, Y. Deng, and K. Li. RAGPulse: An open-source RAG workload trace to optimize RAG serving systems. *arXiv preprint arXiv:2511.12979*, 2025.
- B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin. Fast distributed inference serving for large language models. In *arXiv preprint arXiv:2305.05920*, 2023.
- F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. M. Maben, R. Mehta, W. Chi, L. K. Jang, Y. Xie, S. Zhou, and G. Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. In *Advances in Neural Information Processing Systems 38*, 2025.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- H. Yang, R. Zhang, M. Huang, W. Wang, Y. Tang, Y. Li, Y. Liu, and D. Zhang. KVShare: Semantic-aware KV-Cache sharing for efficient large language model inference. *arXiv preprint arXiv:2503.16525*, 2025b.
- J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37*, 2024.
- J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 94–109. ACM, 2025a.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

- S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan. τ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025b.
- G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pp. 521–538. USENIX Association, 2022.
- W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng. WildChat: 1m ChatGPT interaction logs in the wild. In *The Twelfth International Conference on Learning Representations*, 2024.
- L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems* 38, 2024.

A CORE THEORY AND ALGORITHM NOTATION

This appendix lists only the symbols that define the exact-prefix analysis, the request-level queueing approximation, or PRISM’s core algorithmic decisions. We omit one-off local variables, figure-only labels, and evaluation-only measurements. The two parts are kept separate to avoid overloading: q_i always denotes the i -th request, i is only an index, r always denotes a reusable-segment identity, t denotes a scheduling or admission time index, C_t is reserved for resident KV-cache prefix paths in the theory, and $\mathcal{B}$ variants denote scheduler batches in the algorithm.

A.1 CORE THEORY NOTATION

Table 5: Core notation for exact-prefix analysis and the request-level queueing approximation.

Symbol	Meaning	Role
$q_i, x_i, x^{\text{sys}}, x^{r_i, j}, m_i, x_i^{\text{priv}}$	Request, serialized prompt, reusable-segment count, and system/reusable/private components	Defines the long-prefix request abstraction with i as the request index.
$z_i, C_t, \text{LCP}(z_i, c)$	Token path, resident KV-cache prefix paths, and longest common prefix	Defines exact-prefix KV reuse at admission time.
$L_i^{\text{hit}}(t), L_i^{\text{eff}}(t), \ell(\cdot)$	Exact-prefix hit length, effective prefill length, and token-length function	$L_i^{\text{eff}}(t) = \ell(z_i) - L_i^{\text{hit}}(t)$.
$\text{TFTT}_i, W_i^{\text{admit}}, T_i^{\text{prefill}}, T_i^{\text{1tok}}$	Time to first token and its admission, prefill, and first-token components	Decomposes the latency objective.
$B_t, R_{\text{pf}}(B_t)$	Backend batch state and batch-state-dependent realized prefill throughput	Used only in the request-level prefill-time approximation.
$\pi, h_\pi, \Delta h, \Delta h_{\text{reuse}}$	Serving policy, full-prompt exact-prefix hit rate, full-prompt gap, and reusable-only gap	Δh_{reuse} is converted to the theorem’s token-level Δh by L_{reuse}/L .
$L_\pi, L_{\text{reuse}}, \bar{M}_\pi, R_{\text{pf}, \pi}^{\text{eff}}$	Policy-level prompt length, reusable payload, wave size, and effective prefill throughput	General service-rate decomposition; the theorem is the controlled specialization with fixed $L, \bar{M}$, and $R_{\text{pf}, \pi}^{\text{eff}} = R_{\text{pf}}$.
$S_{\text{wave}}, S_{\text{req}}, c_{s, \pi}^2$	Wave service time, request-level service-time abstraction, and squared coefficient of variation	Used only in the approximate admission-wait model, not in the fixed-parameter theorem.
$\mu_\pi, \lambda, \rho_\pi$	Request-level service rate, offered arrival rate, and utilization	$\mu_\pi = \bar{M}_\pi / \mathbb{E}_\pi[S_{\text{wave}}]$ includes the wave-to-request conversion; $\rho_\pi = \lambda / \mu_\pi$ in the approximate $M/G/1$ abstraction.
$\lambda_\pi^*, \Delta \lambda^*$	Approximate sustainable arrival rate and PRISM’s stability-region expansion	The stability ceiling is request-level and equals μ_π under the approximation.
$W^{\text{floor}}, \rho_\pi^*, \lambda_{\text{crossover}}^*$	Low-load wait floor, crossover utilization, and crossover arrival rate	Used for service-knee calibration of the approximate queueing model.
$\alpha_{\text{dec}, \pi}, X_\pi, L_{\text{out}, \pi}, R_{\text{dec}}$	Decode attenuation, completed throughput, output length, and decode throughput	States the common-attenuation condition; otherwise the effect is absorbed into measured $R_{\text{pf}, \pi}^{\text{eff}}$.

A.2 CORE ALGORITHM NOTATION

Table 6: Core notation for QAS and DART.

Symbol	Meaning	Role
q_i, r, t	Request, reusable-segment identity, and scheduling round	Shared identifiers used across QAS, DART, and the theory.
$\mathcal{R}(q), m_i, \Pi_i, \Pi_i^*$	Reusable-segment set, skeleton length, input skeleton, and service-time skeleton	$\Pi_i^* = \Pi_i$ unless certified reorderable regions are front-aligned.
$\mathcal{Q}^{\text{queue}}, \mathcal{Q}_t^{\text{active}}$	Frozen waiting set and active backend requests	Source sets for queued and active segment demand.
$\Delta t_{\text{win}}, M, q_{\text{cold}}$	Pending-window duration, dispatch budget, and cold-lane quota	Determine the scheduling round and the hot/cold dispatch split.
$g_t(r), a_t(r), n_t(r; \mathcal{B})$	Queued, active, and reference-batch counters for segment r	Raw within-round demand signals.
$P_t(r; \mathcal{B})$	Segment priority under reference batch $\mathcal{B}$	$w_g g_t(r) + w_a a_t(r) + w_n n_t(r; \mathcal{B})$.
w_g, w_a, w_n	Counter weights	Set to 1, 10^6 , and 10^9 in experiments.
$\mathcal{B}_b^{\text{hot}}, n_b(r)$	Provisional hot-lane batch for bucket b and its candidate counter	Used to score each bucket before final dispatch.
$\mathcal{B}_t^{\text{dispatch}}, \hat{P}_t(r)$	Final dispatch batch and dispatch-batch priority	$\hat{P}_t(r) = P_t(r; \mathcal{B}_t^{\text{dispatch}})$ for DART retention.
$f_{\text{front}}, \kappa, \text{Top}_{\kappa}^{\rightarrow}$	Front-alignment width, signature size, and ordered top- κ selector	Define the order-sensitive bucket signature.
$\sigma_i, b_i, \mathcal{T}[b]$	Request signature, bucket ID, and bucket-table entry	Buckets group compatible requests for hot-lane scanning.
$ \mathcal{T}[b] , \mathcal{R}_b$	Raw bucket size and reusable segments represented by bucket b	Size captures request multiplicity; $\mathcal{R}_b$ is the utility support.
$U(b)$	Mean reusable-segment utility of bucket b	Averages $P_t(r; \mathcal{B}_b^{\text{hot}})$ over $\mathcal{R}_b$.
$\text{Score}(b), \alpha_{\text{size}}, \beta_{\text{util}}$	Bucket score and its size/utility weights	$\text{Score}(b) = \alpha_{\text{size}} \mathcal{T}[b] + \beta_{\text{util}} U(b)$.
$u, \text{seg}(u), \text{last}(u)$	Radix node, owning reusable segment, and last-access time	Inputs to DART eviction ordering.
$f, \mathcal{F}_t$	DART protection budget and protected reusable-anchor set	Top- f resident reusable anchors are protected at round t .
$\text{EvictKey}(u)$	Lexicographic eviction key	Private suffixes are reclaimed before low-priority reusable nodes and protected anchors.
K_{free}, V	KV-cache reclaim target and number of evictable radix nodes	Used in eviction pseudocode and the $O(\log V)$ overhead bound.

B ALGORITHM PSEUDOCODE

This appendix provides pseudocode for the two core procedures described in Section 3. In DART, f denotes the protection budget: at each scheduling round, at most f resident reusable anchors are placed in the protected set $\mathcal{F}_t$.

Algorithm 1 QAS: Query-Aware Scheduling

Require: request stream, front-alignment width f_{front} , signature size κ , window duration Δt_{win} , dispatch budget M , cold quota q_{cold}

Ensure: priority-ordered serving queue with per-request KV-cache hints

- 1: Initialize active set $\mathcal{Q}^{\text{active}} \leftarrow \emptyset$ and pending buffer $\mathcal{P} \leftarrow \emptyset$
- 2: **for each** pending window of duration Δt_{win} **do**
- 3: **for each** arriving request q_i with ordered reusable skeleton Π_i **do**
- 4: Append (q_i, Π_i) to $\mathcal{P}$ in arrival order
- 5: **end for**
- 6: **for each** completed in-service request q_j observed before the window freezes **do**
- 7: Remove q_j from $\mathcal{Q}^{\text{active}}$
- 8: **end for**
- 9: Freeze the window and set $\mathcal{Q}^{\text{queue}} \leftarrow \mathcal{P}$
- 10: Compute segment counters $g_t(r)$ from $\mathcal{Q}^{\text{queue}}$ and $a_t(r)$ from $\mathcal{Q}^{\text{active}}$
- 11: Use Eq. (9) to instantiate segment priorities with the appropriate reference batch
- 12: Initialize bucket table $\mathcal{T} \leftarrow \emptyset$
- 13: **for each** $(q_i, \Pi_i) \in \mathcal{P}$ **do**
- 14: $\Pi_i^* \leftarrow$ move the top- f_{front} segments by $P_t(r; \emptyset)$ in reorderable regions to the front, preserving the order of the remaining segments
- 15: $\sigma_i \leftarrow \text{Top}_{\kappa}^{\rightarrow}(\Pi_i^*; P_t(\cdot; \emptyset))$
- 16: $b_i \leftarrow \text{hash}(\sigma_i)$
- 17: Append (q_i, b_i, Π_i^*) to bucket $\mathcal{T}[b_i]$
- 18: Set rank_i to the position of q_i in bucket $\mathcal{T}[b_i]$
- 19: **end for**
- 20: **for each** non-empty bucket b **do**
- 21: $\mathcal{B}_b^{\text{hot}} \leftarrow$ first up to $M - q_{\text{cold}}$ requests from $\mathcal{T}[b]$ in bucket order
- 22: Compute candidate next counter $n_b(r)$ and priorities $P_t(r; \mathcal{B}_b^{\text{hot}})$
- 23: **end for**
- 24: Compute bucket scores $\text{Score}(b)$ and rank buckets in descending order
- 25: Fill up to $M - q_{\text{cold}}$ hot-lane dispatch slots by scanning ranked buckets
- 26: Let $\mathcal{Q}^{\text{cold}}$ be pending requests not yet selected for this dispatch
- 27: Fill the remaining q_{cold} slots with requests in $\mathcal{Q}^{\text{cold}}$ in FIFO order
- 28: Let $\mathcal{B}_t^{\text{dispatch}}$ be the selected dispatch batch and compute actual $n_t(r; \mathcal{B}_t^{\text{dispatch}})$ and $P_t(r; \mathcal{B}_t^{\text{dispatch}})$ from it
- 29: Export per-request hints: bucket/rank, counter snapshot, dispatch priority, and final serialized-prompt offsets
- 30: Remove admitted requests from $\mathcal{P}$ and insert them into $\mathcal{Q}^{\text{active}}$
- 31: **end for**

Algorithm 2 DART: Demand-Aware Eviction**Require:** eviction target size K_{free} , protection budget f , min-heap $\mathcal{H}$ keyed by EvictKey**Ensure:** $\text{freed} \geq K_{\text{free}}$ tokens of KV-cache capacity

```

1: Before eviction, form  $\mathcal{F}_t$  from the top- $f$  resident reusable anchors under the dispatch-batch
   priority
2: EvictKey( $u$ ) is defined by Eq. (13); active and system-critical nodes are excluded from  $\mathcal{H}$ , while
   protected reusable anchor nodes have the lowest reclaim priority
3:  $\text{freed} \leftarrow 0$ 
4: while  $\text{freed} < K_{\text{free}}$  do
5:   if  $\mathcal{H}$  is empty then
6:     Collect evictable nodes into  $\mathcal{H}$ 
7:   end if
8:   if  $\mathcal{H}$  is empty then
9:     break
10:  end if
11:   $(\text{key}_{\text{stored}}, u) \leftarrow \text{HEAPPUSH}(\mathcal{H})$ 
12:  if  $u$  is no longer an attached evictable node then
13:    continue
14:  end if
15:   $\text{key}_{\text{current}} \leftarrow \text{EvictKey}(u)$ 
16:  if  $\text{key}_{\text{stored}} \neq \text{key}_{\text{current}}$  then
17:     $\text{HEAPPUSH}(\mathcal{H}, \text{key}_{\text{current}}, u)$ 
18:    continue
19:  end if
20:  Detach  $u$  from the radix tree and reclaim its  $|u|$  KV blocks
21:   $\text{freed} \leftarrow \text{freed} + |u|$ 
22:  if  $\text{parent}(u)$  is now evictable then
23:     $\text{HEAPPUSH}(\mathcal{H}, \text{EvictKey}(\text{parent}(u)), \text{parent}(u))$ 
24:  end if
25: end while

```

B.1 IMPLEMENTATION NOTES

QAS computes all segment counters after the pending window freezes, so alignment and bucket construction use the current scheduling state rather than a stale arrival-time snapshot. The candidate next counter $n_b(r)$ is a one-shot counterfactual over already materialized pending requests; it is not a prediction of future arrivals and is not recomputed while the final dispatch scans multiple buckets. The cold lane admits the oldest unselected requests in FIFO order, which prevents rare-prefix requests from waiting indefinitely.

Because radix splits and detachments can stale heap entries, DART validates keys lazily at pop time and reinserts stale entries before reclaiming blocks. Section 3.4 summarizes the control-plane complexity.

C HYPERPARAMETER SENSITIVITY

C.1 HOT REQUEST RATE

Setup. We vary the hot-request rate over $r_{\text{hot}} \in \{0.5, 0.3, 0.1\}$ on Qwen3-4B-Instruct-2507 while keeping the same Zipf exponent, hotspot set size, Poisson arrivals, replay length, and QPS sweep as the main benchmark. Lower r_{hot} reduces the number of requests that touch hotspot segments, so the experiment directly tests how much PRISM depends on reusable-prefix mass.

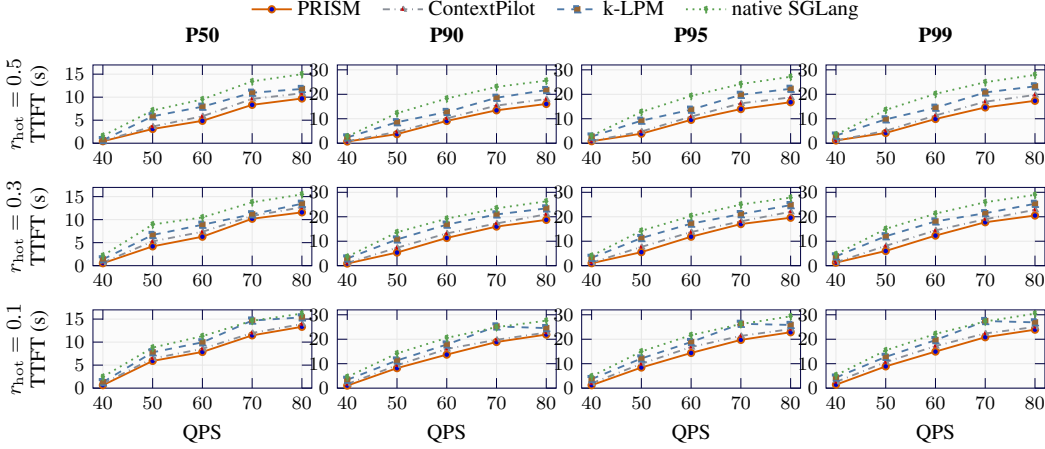


Figure 7: TTFT percentile sensitivity to the hot-request rate on Qwen3-4B-Instruct-2507. Rows vary r_{hot} , and columns report P50, P90, P95, and P99 TTFT over the same QPS sweep.

Figure 7 shows the full TTFT distribution under decreasing reusable-prefix overlap. PRISM improves not only the tail but also the middle of the distribution: across the 50–80 QPS stress region, its P50 remains below the strongest non-PRISM baseline by 10.4–16.9%, 5.7–20.6%, and 4.2–8.2% for $r_{\text{hot}} \in \{0.5, 0.3, 0.1\}$, respectively. The gap is largest when the workload still has enough hot segments for coordinated batching and retention to reshape the realized prefill waves.

The P90–P99 panels follow the same mechanism. At the lightest load, queueing is small and the methods can be nearly tied; for example, at $r_{\text{hot}}=0.5$ and 40 QPS, ContextPilot is only 0.01 s lower in P99 TTFT. Once the offered load reaches 50–80 QPS, PRISM reduces P99 TTFT by 13.8%, 14.3%, and 10.2% on average at $r_{\text{hot}} \in \{0.5, 0.3, 0.1\}$, respectively, relative to the best non-PRISM baseline at each load. The corresponding average KV-cache hit rates, although omitted from the figure to keep the percentile view complete, decrease from 42.1% to 37.2% and 32.7% as r_{hot} falls, while PRISM still leads the strongest non-PRISM KV-cache baseline by 4.72–8.45 percentage points across the sweep.

C.2 ZIPF EXPONENT

Setup. We additionally run a fixed-load single-seed sweep on Qwen3-4B-Instruct-2507 at 60 QPS with $r_{\text{hot}}=0.5$, varying the Zipf exponent over $\alpha \in \{0.6, 0.8, 1.2, 1.5, 2.0\}$. This isolates how popularity concentration changes the value of prefix coordination without conflating the result with a separate load transition. Section 4.2 describes the workload construction and visualizes how the Zipfian hotspot process matches the short-window skew observed in real RAG traces.

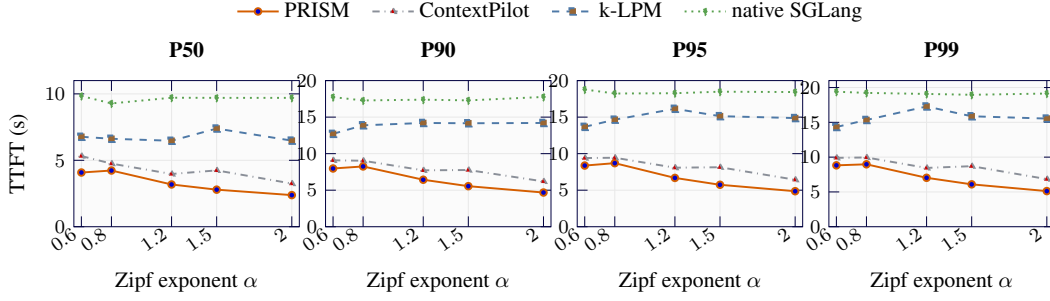


Figure 8: Fixed-load TTFT percentile sensitivity to the Zipf exponent α on Qwen3-4B-Instruct-2507 (60 QPS, $r_{\text{hot}}=0.5$).

Figure 8 shows the full TTFT distribution as the hotspot popularity becomes more concentrated. PRISM remains the lowest-latency system at every reported percentile and every α . Relative to the strongest non-PRISM baseline at each point, it reduces P50 TTFT by 10.9–34.2% and P99 TTFT by 9.8–30.0%. The absolute tail also improves with stronger skew: PRISM’s P99 TTFT decreases from 8.82 s at $\alpha=0.6$ to 5.12 s at $\alpha=2.0$.

The KV-cache metrics move in the same direction. PRISM’s backend exact-prefix hit rate rises from 47.1% to 51.1% as α increases from 0.6 to 2.0, while ContextPilot, the strongest non-PRISM KV-cache baseline, rises from 41.4% to 46.9%. PRISM maintains a 4.20–6.51 percentage-point KV-cache-hit advantage across the sweep. This supports the skew story in Section 2: when reusable-prefix mass concentrates on fewer hot segments, scheduler–KV-cache co-design can turn more of that mass into realized prefill reuse.

D TRACE VALIDATION AND CONFIGURATION DISCLOSURE

D.1 MAIN-EXPERIMENT CONFIGURATION DISCLOSURE

Throughout the paper, we describe the MultiHopRAG setup using a normalized budget of $k=5$ with nominal 128-token chunks. We therefore use the observed prompt lengths, rather than literal budget arithmetic, when instantiating Section 2.4 and Appendix E.

In the Qwen3-4B-Instruct-2507 $r_{\text{hot}}=0.7$ sweep, the materialized request statistics are mean/p50/p90/p99/max = 768.1/770/791/818/879 prompt tokens for $k=5$, 1029.5/1036/1059/1083/1147 for $k=7$, and 1429.6/1438/1461/1486/1548 for $k=10$. The $k=5$ backend KV-cache accounting is consistent with this materialization: across the PRISM sweep it reports about 771–773 total prompt tokens per accounted request and about 660–662 document-prefix tokens per request. The traced 13B validation path remains mean/p50/p90/p99/max = 371.5/370/417.5/457/485 tokens, with mean reusable prefix 295.3 tokens and mean private suffix 76.1 tokens. On the 13B path, no request exceeds `max_model_len=4096` and no runtime prompt truncation occurs. This disclosure is important for interpreting the queueing analysis correctly: the normalized $k=5$, 128-token description is a workload budget, whereas the instantiated service demand uses the realized prompt lengths that the executable pipeline actually produced.

D.2 TTFT MEASUREMENT SEMANTICS

In the 4B phase traces, TTFT is reported as frontend *pre-admission* plus an *admission-to-first-token* envelope. The first field measures request submission through dispatcher admission. The second field includes backend enqueueing, any GPU-service wait, chunked-prefill execution, and the first-token return path. It is therefore an upper envelope for $T_i^{\text{prefill}} + T_i^{\text{tok}}$ rather than a pure prefill timer. We use these traces for the service-knee calibration in Figure 5; the older 13B request-level trace remains the finer validation source for the isolated prefill-timer decomposition.

D.3 EMPIRICAL SERVICE-KNEE CALIBRATION ON A SINGLE A800

Table 7 summarizes the Qwen3-4B-Instruct-2507 $k=5$, $r_{\text{hot}}=0.7$ PRISM sweep. The table uses directly observed quantities from the exported CSV and backend JSON: sustained throughput, P99 TTFT, backend document-prefix hit rate, average prefill-wave size, average uncached/extend tokens per wave, and the mean admission-to-first-token envelope.

Table 7: Qwen3-4B-Instruct-2507 PRISM service-knee calibration on a single A800 ($k=5$, $r_{\text{hot}}=0.7$). The admission-to-first-token envelope includes backend queueing, prefill, and first-token return.

Target QPS	Throughput (req/s)	P99 TTFT (s)	Doc. hit (%)	$\bar{M}$	Extend/wave (tokens)	Admit→1tok mean (s)
40	38.72	0.86	48.98	6.42	2.63k	0.29
50	48.28	2.00	48.90	23.87	9.80k	0.93
60	49.79	6.50	49.25	33.11	13.51k	3.48
70	50.44	10.99	49.10	34.25	14.00k	6.18
80	50.06	14.14	49.27	33.74	13.73k	8.10

The knee is visible in both throughput and tail latency. At 40 and 50 offered QPS, throughput tracks the offered load and P99 TTFT remains below 2.01 s. From 60 to 80 offered QPS, throughput plateaus around 50 req/s while P99 TTFT grows from 6.50 s to 14.14 s, so we treat 50–60 QPS as the empirical transition interval for this 4B configuration. The top- k stress sweep gives the same directional check predicted by Eq. (5): at 40 offered QPS, increasing k raises mean prompt length from 768.1 to 1029.5 and 1429.6 tokens, while PRISM throughput drops from 38.72 to 28.93 and 21.07 req/s and P99 TTFT rises from 0.86 s to 18.55 s and 44.97 s. Larger L lowers the sustainable service rate even though PRISM still realizes the highest exact-prefix hit rate among the compared systems.

For the Llama2-13B validation trace, the earlier request-level instrumentation still places the prefill-to-congestion crossover between 12 and 15 QPS. After subtracting the 5-QPS batching floor, the 10- and 12-QPS points remain prefill-dominated, whereas 15 QPS is already congestion-dominated. We therefore treat both reported intervals as workload- and hardware-specific single-A800 calibrations, not as universal constants.

E PROOFS FOR THE TWO-STAGE QUEUEING MODEL

This appendix gives the queueing approximation used by Section 2.4, the proof of Theorem 1, the crossover formula, and the calibration used to interpret batch-state and service-time variability.

E.1 POLICY-DEPENDENT DECOMPOSITION AND PROOF

General decomposition. Eq. (5) gives the request-level service rate under policy π as

$$\mu_{\pi} = \frac{\bar{M}_{\pi}}{\mathbb{E}_{\pi}[S_{\text{wave}}]} = \frac{\bar{M}_{\pi} R_{\text{pf},\pi}^{\text{eff}}}{L_{\pi}(1 - h_{\pi})}.$$

For any two policies π_1, π_0 , direct division yields Eq. (6). This form keeps batch composition, effective hardware throughput, prompt length, and exact-prefix reuse separate. The fixed-parameter theorem in the main text is the specialization that isolates the last factor.

Fixed-parameter service-rate gap (Eq. (7)). Holding L , $\bar{M}$, and the measured effective R_{pf} fixed across PRISM and LRU, Eq. (5) reduces to

$$\mu_{\pi} = \frac{\bar{M} R_{\text{pf}}}{L(1 - h_{\pi})}, \quad \pi \in \{\text{PRISM}, \text{LRU}\},$$

so μ_{π} carries units of requests/s (the wave conversion contributes a factor of $\bar{M}$ in the numerator because each wave releases $\bar{M}$ completions in $\mathbb{E}[S_{\text{wave}}]$ seconds). Using $1 - h_{\text{PRISM}} = (1 - h_{\text{LRU}}) -$

Δh with $\Delta h \geq 0$,

$$\begin{aligned}\mu_{\text{PRISM}} - \mu_{\text{LRU}} &= \frac{\bar{M} R_{\text{pf}}}{L} \left(\frac{1}{1 - h_{\text{LRU}} - \Delta h} - \frac{1}{1 - h_{\text{LRU}}} \right) \\ &= \frac{\bar{M} R_{\text{pf}}}{L} \cdot \frac{\Delta h}{(1 - h_{\text{LRU}} - \Delta h)(1 - h_{\text{LRU}})} \\ &= \mu_{\text{LRU}} \cdot \frac{\Delta h}{1 - h_{\text{LRU}} - \Delta h} = \mu_{\text{LRU}} \cdot \frac{\Delta h}{1 - h_{\text{PRISM}}},\end{aligned}$$

which is Eq. (7). Conditional non-negativity follows from $\Delta h \geq 0$ and $h_{\text{PRISM}} < 1$, and equality holds iff $\Delta h = 0$. $\square$

Stability expansion (Eq. (8)). Because λ and μ_π are both in requests/s, the $M/G/1$ approximation is stable iff $\rho_\pi = \lambda/\mu_\pi < 1$, so the approximate request-level sustainable arrival rate is $\lambda_\pi^* = \mu_\pi$. Hence $\Delta\lambda^* = \mu_{\text{PRISM}} - \mu_{\text{LRU}}$, which is non-negative by the first part of the theorem.

Substituting the token-level hit-rate gap directly into the middle line of the display above gives

$$\Delta\lambda^* = \frac{\bar{M} R_{\text{pf}}}{L} \cdot \frac{\Delta h}{(1 - h_{\text{LRU}} - \Delta h)(1 - h_{\text{LRU}})},$$

which is Eq. (8). $\square$

If a result table reports reuse only over the reusable portion of the prompt, let Δh_{reuse} be that reusable-token hit-rate gap and let L_{reuse} be the mean reusable-token payload. The full-prompt token gap used in the queueing expression is then $\Delta h = \Delta h_{\text{reuse}} L_{\text{reuse}}/L$. We use the full-prompt form in the theorem to avoid conflating segment counts with token-weighted KV-cache savings.

Wave-to-request conversion and the $\bar{M}$ factor. The correct conversion from wave-level to request-level throughput is central to the derivation above, and it is worth isolating. The natural first attempt at a single-server abstraction is to write $\mu_\pi^{\text{wave}} := 1/\mathbb{E}[S_{\text{wave}}] = R_{\text{pf}}/[L(1 - h_\pi)]$, which has units of waves per second. Feeding a request-level arrival rate λ (requests per second) into $\rho = \lambda/\mu_\pi^{\text{wave}}$ produces a quantity that is dimensionally incorrect by a factor of $\bar{M}$ and understates sustainable λ^* by that same factor. The correct relation is $\mu_\pi = \bar{M} \mu_\pi^{\text{wave}}$ because each wave releases $\bar{M}$ completions. Numerically, the Qwen3-4B-Instruct-2507 $k=5$ trace at 60 QPS has $\bar{M} \approx 33.1$, $L \approx 768$, and a full-prompt token hit rate $h_{\text{PRISM}} \approx 0.474$ (document-prefix hit rate 49.25%). Calibrating the effective per-request prefill throughput from the sustained $\mu_{\text{PRISM}} \approx 49.8$ req/s gives $R_{\text{pf}} \approx 6.1 \cdot 10^2$ effective tokens/s/request. Reading the same service process as waves/s would give only $\mu_{\text{PRISM}}/\bar{M} \approx 1.5$ waves/s, off by the batch-size factor.

E.2 ADMISSION-WAIT APPROXIMATION

We approximate the bulk-service backend by an $M/G/1$ server with request-level service rate μ_π . Fed by Poisson arrivals of rate λ , the utilization is $\rho_\pi = \lambda/\mu_\pi$, and the admission-wait approximation used in the main text is

$$\mathbb{E}[W^{\text{admit}}] = \frac{1}{2} \Delta t_{\text{win}} + \frac{\rho_\pi}{1 - \rho_\pi} \cdot \frac{1 + c_{s,\pi}^2}{2} \cdot \frac{1}{\mu_\pi}, \quad c_{s,\pi}^2 = \frac{\text{Var}(S_{\text{req}})}{(\mathbb{E}[S_{\text{req}}])^2}. \quad (14)$$

The first term is a wave-alignment floor, and the second term is the $M/G/1$ queueing term. This compact approximation preserves the heavy-traffic blow-up as $\rho_\pi \rightarrow 1$ while abstracting away the full bulk-service state.

E.3 PROOF OF THE CROSSOVER FORMULA

We derive Eq. (15). Let $W^{\text{floor}} := \widehat{W}_{\text{floor}}^{\text{batch}}$ absorb the wave-alignment term $\frac{1}{2} \Delta t_{\text{win}}$ and other low-load overhead, so that the congestion-sensitive part of Eq. (14) is

$$\mathbb{E}[W^{\text{admit}}] - W^{\text{floor}} = \frac{\rho_\pi}{1 - \rho_\pi} \cdot \frac{1 + c_{s,\pi}^2}{2} \cdot \frac{1}{\mu_\pi}.$$

The idealized crossover condition used for calibration sets this equal to $\mathbb{E}[T^{\text{prefill}}]$. Because all $\bar{M}$ requests in a wave prefill in parallel, a request's experienced prefill time equals the wave duration, $\mathbb{E}[T^{\text{prefill}}] = \mathbb{E}[S_{\text{wave}}] = \bar{M}/\mu_\pi$ (the wave releases $\bar{M}$ completions per S_{wave} , so $\mu_\pi = \bar{M}/\mathbb{E}[S_{\text{wave}}]$). Substituting,

$$\frac{\rho_\pi}{1 - \rho_\pi} \cdot \frac{1 + c_{s,\pi}^2}{2} \cdot \frac{1}{\mu_\pi} = \frac{\bar{M}}{\mu_\pi} \iff \frac{\rho_\pi^*}{1 - \rho_\pi^*} = \frac{2\bar{M}}{1 + c_{s,\pi}^2}.$$

Solving the linear equation for ρ_π^* in $(0, 1)$ gives

$$\rho_\pi^* = \frac{2\bar{M}}{1 + c_{s,\pi}^2 + 2\bar{M}}.$$

Multiplying by μ_π yields

$$\lambda_\pi^* = \rho_\pi^* \mu_\pi = \mu_\pi \cdot \frac{2\bar{M}}{1 + c_{s,\pi}^2 + 2\bar{M}}. \quad (15)$$

□

Qualitative behavior. Eq. (15) has three consequences. (i) $\lambda_\pi^* \rightarrow \mu_\pi$ as $\bar{M} \rightarrow \infty$: larger batches push the crossover toward the stability ceiling, consistent with the intuition that batching defers congestion. (ii) $\lambda_\pi^* \rightarrow \mu_\pi \cdot 2/(1 + c_{s,\pi}^2 + 2)$ as $\bar{M} = 1$: for $c_{s,\pi}^2 = 1$ this reduces to $\mu_\pi/2$, recovering the classical unbatched $M/M/1$ knee at $\rho = 1/2$. (iii) The crossover is monotonically increasing in μ_π , so the *shift* in crossover between PRISM and a KV-cache-oblivious baseline follows Theorem 1: $\Delta\lambda_{\text{crossover}}^* \approx (2\bar{M}/(1 + c_{s,\pi}^2 + 2\bar{M})) \Delta\lambda^*$.

Numerical consistency with Appendix D. On the Qwen3-4B-Instruct-2507 $k=5$ sweep, PRISM plateaus near $\mu_{\text{PRISM}} \approx 50.1$ req/s over the 60–80 QPS offered-load points. Using the 60-QPS backend accounting $\bar{M} \approx 33.1$, Eq. (15) gives $\lambda_{4B}^* \approx 48.6$ QPS when $c_{s,\pi}^2 = 1$, at the lower edge of the observed 50–60 QPS transition in Table 7. Varying $c_{s,\pi}^2$ over $[0.5, 4]$ gives a narrow 46.6–49.0 QPS interval because $2\bar{M}$ dominates the denominator. On Llama2-13B the sustained rate is $\mu_{\text{PRISM}} \approx 14.5$ req/s in the 15-QPS operating regime; with $\bar{M} \approx 9.75$, Eq. (15) gives 13.1 QPS at $c_{s,\pi}^2 = 1$ and 11.5–13.5 QPS over the same sensitivity range, consistent with the 12–15 QPS transition. These are interval-level predictions; a per-QPS match would require wave-level service moments and underfill distributions from the traces.

E.4 DECODE CONTENTION AND THE COMMON-ATTENUATION ASSUMPTION

Decode contention can be written as a multiplicative attenuation of prefill throughput:

$$R_{\text{pf},\pi}^{\text{eff}} = R_{\text{pf}}(1 - \alpha_{\text{dec},\pi}), \quad \alpha_{\text{dec},\pi} = \frac{X_\pi L_{\text{out},\pi}}{R_{\text{dec}}}, \quad (16)$$

where X_π is completed request throughput, $L_{\text{out},\pi}$ is mean output length, and R_{dec} is decode throughput. Here we state when that factor can be treated as common across policies and when it must instead be absorbed into the measured effective prefill rate.

Steady-state invariance. Under stationary operation with arrival rate λ and serving policies $\pi \in \{\text{PRISM}, \text{LRU}\}$ that sustain the offered load (i.e. $\rho_\pi < 1$), completed throughput equals λ for both policies. If, in addition, (i) mean output length is determined by the prompt-and-model rather than by prefix KV-caching, and (ii) decode throughput is approximately a hardware-and-model constant under the compared batch states, then $X_\pi \approx \lambda$, $L_{\text{out},\pi} \approx L_{\text{out}}$, and Eq. (16) yields a common attenuation factor. In that stable regime the factor cancels in the service-rate gap, leaving h_π as the policy-dependent term.

Empirical use in our traces. The KV-cache-policy ablation in Section 4.5 keeps the workload and serving configuration fixed while varying only the backend retention rule. Across 60–80 QPS on Qwen3-4B-Instruct-2507, PRISM realizes higher exact-prefix hit rate and lower P99 TTFT than LRU, LRU with an active-demand counter, and LFU. We therefore use this ablation as evidence for the scheduler–KV-cache feedback loop rather than as a claim that the per-token prefill cost is analytically identical across all backend states. Any residual variation in realized per-token cost is absorbed into the measured effective R_{pf} used for crossover calibration.

When common attenuation breaks. Three regimes can violate assumption (ii) and reintroduce a policy-dependent decode term. (a) When output length is policy-sensitive—e.g. different answer quality leading to different L_{out} —the Little’s-law cancellation fails; this is not our setting. (b) When chunked-prefill chunk size is co-tuned with concurrent decode (as in certain Sarathi-style schedulers), a policy that admits larger waves may crowd out decode slots and slow completion, inflating the in-flight request load $\lambda_{\text{in-flight}}$ and hence $\alpha_{\text{dec},\pi}$; our 4B and 13B traces use a fixed chunk configuration across policies. (c) When decode throughput is memory-bandwidth-limited and prefill is FLOP-limited (decode-bound regime with very long outputs), R_{dec} can degrade as prefill concurrency grows; this is negligible in our short-output RAG workload (L_{out} is a short answer span) but would need to be modeled explicitly for output-heavy workloads such as long-form generation. These regimes are outside the scope of the closed-form Theorem 1.

E.5 REMARK ON SERVICE-TIME VARIABILITY

Theorem 1 establishes that a policy has a larger μ_π whenever it realizes a larger token-level exact-prefix hit rate under the fixed-parameter comparison. Eq. (6) is the corresponding policy-dependent decomposition when batch size or effective throughput differs. The second moment of service time can further affect admission wait through Eq. (14), but we do not rely on a formal variance-contraction theorem.

The KV-cache-policy ablation in Section 4.5 shows the same qualitative effect under fixed serving conditions. Relative to the strongest non-PRISM hit-rate baseline, PRISM raises KV-cache hit rate by 18.54–18.86 percentage points; relative to the best non-PRISM P99 policy at each offered load, it reduces P99 TTFT by 37.6%, 15.9%, and 25.4% at 60, 70, and 80 QPS. This observation is consistent with PRISM improving both average service and service regularity, but the trace evidence is used qualitatively; a precise claim about $c_{s,\pi}^2$ would require computing wave-level service-time moments directly from the trace.

A rigorous variance result would require assumptions on the dependence structure of hit indicators within a wave and on the backend’s chunked-prefill service process. We leave that formalization to future work and keep the closed-form theorem focused on the hit-rate-induced service-rate expansion, which is the part directly supported by algebra and by the measured hit-rate and effective-token differences.

F VARYING PROMPT LENGTH

This appendix evaluates whether PRISM remains effective when prompt length changes. We modify the MultiHopRAG workload by varying retrieval top- k over $\{7, 10, 15\}$ while keeping the serving setup fixed to Qwen3-4B-Instruct-2507 on the single A800 GPU at 60 QPS. All methods use the same arrival trace, hotspot construction, and empty KV-cache start; only the number of retrieved passages per request changes. We report exact-prefix backend KV-cache hit rate and P99 TTFT, the two metrics most directly tied to scheduler–KV-cache coordination.

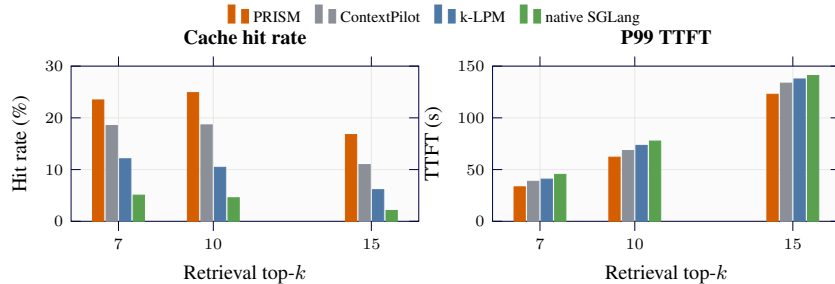


Figure 9: MultiHopRAG retrieval top- k sensitivity on Qwen3-4B-Instruct-2507 at 60 QPS. The left panel reports exact-prefix KV-cache hit rate, and the right panel reports P99 TTFT.

Figure 9 shows that increasing top- k raises serving pressure for every method: P99 TTFT grows from the $k=7$ setting to the $k=15$ setting as each request carries more retrieved context. PRISM

nevertheless preserves the highest KV-cache hit rate at every top- k , reaching 23.46%, 24.87%, and 16.76% for $k=7, 10, 15$, respectively. The strongest KV-cache-hit baseline is ContextPilot, at 18.51%, 18.64%, and 10.96%, so PRISM maintains a 4.95–6.23 percentage-point advantage as retrieval depth increases.

The latency trend follows the same mechanism. PRISM gives the lowest P99 TTFT at all three retrieval depths: 33.32 s at $k=7$, 62.03 s at $k=10$, and 122.79 s at $k=15$. Relative to the strongest baseline at each top- k , PRISM reduces P99 TTFT by 13.8%, 9.3%, and 8.1%, respectively. Thus, even when larger retrieval budgets increase the absolute amount of prefill work and reduce the attainable reuse fraction, scheduler-informed retention still converts more repeated evidence context into realized exact-prefix hits.